

Passenger-flow Interval Aggregate Uplink Protocol

Device → your server. The device actively POSTs; you only need to implement the receiving endpoint.

This protocol reports, once per minute, passenger-flow statistics that are already computed: how many people entered this minute, how many of them were repeats, how many were staff/riders, how age/gender/height are distributed, and how long they stayed. Receiving only this one message gives you every number the report needs — no aggregation on your side.

0. Integration Checklist

Implementing the receiver needs only the four things below; the rest of the document explains fields and edge cases, consult as needed.

Purpose	Method & default path	Content-Type
Interval-aggregate message	POST /api/passenger-flow/interval-aggregate	application/json

The path and server address are both configurable on the device side, so they can adapt to a gateway prefix.

1. Return 2xx **and** a non-empty response body (even just `{}`). The device treats "2xx but empty body" as failure and resends — a reverse proxy or gateway may return an empty 200 when the backend never received the request, and checking status code alone would mark lost data as delivered.
2. One message = one minute; replace the whole record in storage by key `(master\_sn, batch\_date, start\_ts\_s)`. No field-level merge, no accumulation.
3. The same minute may arrive multiple times; resolve with `revision`: keep only the message with the larger `revision`. Discard the smaller one, but still return 2xx + non-empty body.
4. All writes must be idempotent: duplicate delivery will happen; replaying any number of times yields the same final state.

On receiving a message:

```
key = (master_sn, batch_date, start_ts_s)
if key not in DB: insert
elif message.revision > DB_row.revision: overwrite
else: discard
return 200 {"ok": true}
```

Statistical query: just query directly, no extra filter conditions.

That's all. No pointer table, no batch state to maintain, no second message format, no extra conditions when querying.

⚠ When an on-site device is reset, or its installation config changes, the numbers already reported for that day are cleared minute-by-minute by sending one all-zero message per minute (see 3.2). They look exactly like ordinary messages — handle them by the rules above; you do not need any special branch for this.

1. General Conventions

1.1 Time and Time Zone

All timestamps are 10-digit Unix seconds (UTC epoch seconds), with the `\_s` suffix on the field name. The protocol has no millisecond field.

`timezone` is the device's local time zone, used to render timestamps as local time. All three keys are always present; when the device cannot obtain a time zone the whole object is empty `{}`.

Key	Meaning
name	Authoritative value — only the time-zone name lets you recompute a historical day's DST rules
utc_offset_seconds	The offset at the moment the device sent this message (east positive), in seconds; lets a server without a time-zone library render local time directly
is_dst	Whether this moment is in daylight saving time

⚠ `name` is not guaranteed to be a standard time-zone name. The device may be set directly by offset, in which case `name` looks like `"UTC-12:45"`. Accept both forms; do not discard the message just because time-zone-library parsing failed.

⚠ The offset is a snapshot, not a constant of that device. In DST regions the same device reports two different `utc\_offset\_seconds` within a year; recomputing history must go through `name`.

1.2 batch_date — Business Day

The business day the data belongs to (YYYY-MM-DD), divided by the "day-cut time" configured on the device, and may cross midnight: with day-cut set to 02:00, traffic at 01:00 still belongs to the previous day.

⚠ `start\_ts\_s` is unrelated to the calendar day of `batch\_date`. A business day may cross midnight; a segment whose `batch\_date` is 2026-03-04 can easily have its `start\_ts\_s` fall on the calendar day 03-05. Do not reverse-engineer `batch\_date` from `start\_ts\_s`, and do not use `batch\_date` to validate whether `start\_ts\_s` is "reasonable".

1.3 device_sns — Current Device List

All cameras currently installed in this store (including the master camera itself). Multiple cameras in one store jointly produce one set of statistics; this field says which cameras contributed this number. After adding or removing cameras, the list in subsequent messages changes accordingly.

1.4 How to Read the Buckets

The statistics carry bucket details below them (age/gender, height band, person type, dwell duration). The dimension keys are always present and never missing. Three special bucket names must be kept distinct:

Bucket name	Meaning	How to handle
not_measured	This detection was switched off when these people passed	Count into the total, in its own cell; do not merge into unknown
unknown	Detection was on, but could not be recognized (blurry face, model gave no conclusion)	Count into the total, as unknown
other bucket names	Normal recognition result	Count normally
empty array	No people to count this minute	A normal 0

⚠ Keep `not\_measured` and `unknown` strictly separate. Merging them makes it impossible to answer "how many people today were truly unrecognized" — your detection coverage would count "feature off" as "model failed to recognize".

Within the same day, `not\_measured` and normal buckets can appear together (detection toggled mid-day); this is normal.

2. Message

One message = one minute. The interval width is fixed at 60 seconds.

2.1 One Complete Message

```
{
  "schema_version": "1.0",
  "type": "interval_aggregate",
  "master_sn": "CAM-A01",
  "device_sns": ["CAM-A01", "CAM-B02"],
  "timezone": { "name": "Asia/Shanghai", "utc_offset_seconds": 28800, "is_dst":
false },
  "batch_date": "2026-03-04",
  "stat_basis": "enter",
  "revision": 1741046520,
  "interval_s": 60,
  "start_ts_s": 1741046400,
  "end_ts_s": 1741046460,
  "event_counts": { "enter": 7, "leave": 4, "pass_by": 2, "turn_back": 0 },
  "raw_stats": {
    "count": 7,
    "breakdowns": {
      "height_category": [
        { "height_category": "above_threshold", "count": 5 },
        { "height_category": "below_threshold", "count": 1 },
        { "height_category": "unknown", "count": 1 }
      ],
      "age_range|gender": [
        { "age_range": "17_30", "gender": "male", "count": 3 },
```

```

        { "age_range": "17_30", "gender": "female", "count": 2 },
        { "age_range": "31_45", "gender": "male", "count": 1 },
        { "age_range": "unknown", "gender": "female", "count": 1 }
      ]
    },
    {
      "final_stats": {
        "count": 4,
        "repeat": 1,
        "breakdowns": {
          "height_category": [
            { "height_category": "above_threshold", "count": 3 },
            { "height_category": "below_threshold", "count": 1 }
          ],
          "age_range|gender": [
            { "age_range": "17_30", "gender": "male", "count": 2 },
            { "age_range": "17_30", "gender": "female", "count": 1 },
            { "age_range": "31_45", "gender": "male", "count": 1 }
          ]
        }
      },
      "non_customer": {
        "count": 2,
        "breakdowns": {
          "person_type": [
            { "person_type": "staff", "count": 1 },
            { "person_type": "rider", "count": 1 }
          ]
        }
      },
      "dwell_stats": {
        "count": 3,
        "breakdowns": {
          "dwell_range": [
            { "dwell_range": "1_5", "count": 2 },
            { "dwell_range": "5_15", "count": 1 }
          ]
        }
      }
    }
  ]
}

```

What happened this minute: 7 entries at the door. These 7 split into three categories — 4 are first-time customers today, 1 is a repeat (came earlier today), 2 are staff/rider. Also, 3 of the entries already have a corresponding exit, so dwell time can be computed.

$7 = 4 + 1 + 2$ — this addition holds in every message and can be used directly for ingestion validation.

2.2 Envelope Fields

Field	Type	Meaning
schema_version	string	Always ``1.0``
type	string	Always ``interval_aggregate``
master_sn	string	Master camera SN. One per store; used to tell stores apart

device_sns	string[]	Current device list, see 1.3
timezone	object	Device's local time zone, see 1.1
batch_date	string	Business day, see 1.2
stat_basis	string	`enter` or `leave` — the statistical basis direction, see below
revision	int64	The moment this version was computed; only compare magnitudes, see 3.1
interval_s	int	Interval width (seconds), always 60
start_ts_s	uint64	Segment start, inclusive
end_ts_s	uint64	Segment end, exclusive; always = start_ts_s + 60

`stat\_basis` — statistical basis direction: limited by installation angle, a camera can usually only capture a person's front from one direction. The side it can capture the front from is the reliable observation, so it becomes the statistical basis. All cameras in a store share one basis direction, decided at installation and unchanged during normal operation; if it really must change on site, the device treats it as a reset (see 3.2).

⚠ The last four statistics blocks count only the basis-direction people. The opposite direction appears only in `event\_counts`.

⚠ Do not put `stat\_basis` in the primary key; treat it as a descriptive field on the row. Once the basis direction changes, the device clears and recomputes the day (see 3.2), and the new numbers overwrite the old under the same key — two directions' current data can never coexist. Putting it in the key would make the old and new versions after a direction change occupy separate rows and be double-counted.

How segments are cut: `start\_ts\_s` is always aligned to the whole minute. Segments are time-grid cells; not every cell is guaranteed to have data, nor to be contiguous — if no one passes this minute, there is simply no message (see 4.2).

2.3 The Five Statistics Blocks

Look at the big picture first. The same minute's people are counted from five angles by these five blocks:

Block	Counts	Unit	Includes	One-liner
event_counts	How many times each of the four door directions fired	count	everyone	rawest in/out volume
raw_stats	Basis-direction entries	count	everyone (incl. staff/rider)	un-deduplicated flow
final_stats	customers	persons + repeat count	customers only	precise flow
non_customer	staff / rider /	count	non-customers	what to subtract

	courier		only	from flow
dwell_stats	how long customers stayed	count	customers who left store only	dwell distribution

The middle three blocks partition the same group of people without overlap or duplication, so always:

```
raw_stats.count = final_stats.count + final_stats.repeat + non_customer.count
7 = 4 + 1 + 2
```

7 entries → 4 new customers, 1 repeat (came earlier today), 2 staff/rider.

`event\_counts` and `dwell\_stats` do not participate in this addition: the former also includes the opposite direction and pass-by/turn-back, the latter counts only those who have been seen leaving.

① event_counts — raw door in/out (headcount)

How many times each of the four directions fired this minute; un-deduplicated, includes staff/rider, all four directions reported.

Key	Meaning
enter	entered
leave	left
pass_by	passed by (appeared in the door zone but did not enter)
turn_back	turned back (crossed the door line then retreated)

All four keys are always present; 0 means this direction genuinely had no one this minute.

⚠ Pass-by and turn-back appear only here — these people did not enter, and are absent from the other four blocks.

⚠ Right after reporting, `event\_counts[stat\_basis]` may be slightly larger than `raw\_stats.count`, then catch up. Do not use their equality as a validation condition (see 2.5).

② raw_stats — raw headcount (un-deduplicated)

On the basis direction, every entry this minute counts once; the same person entering twice counts twice, and staff/rider are also counted. This is the usual "raw flow".

Field	Meaning
count	Total basis-direction headcount this segment
breakdowns	Attribute distribution, classified by the recognition result of each entry

Attribution rule: each entry is attributed to the minute of its own occurrence time.

③ final_stats — customers (after deduplication)

Field	Unit	Meaning
count	persons	Number of first-time customers this segment — this is the "precise flow"

repeat	headcount	Repeat headcount this segment (came today, came again)
breakdowns		Attribute distribution, covering only the `count` group

Attribution rule: a customer is attributed to the single unique segment of their earliest entry that day, counted once there; each subsequent return is counted into that return's segment's `repeat` by its own time.

⚠ `count` is persons, `repeat` is headcount — different units, do not add them directly as persons.

⚠ The sum of `breakdowns` buckets = `count`, excluding `repeat`. Attributes belong to the person; a second visit by the same person does not re-count attributes.

⚠ Deduplication scope is "one business day + all cameras of this store". Someone who came yesterday and comes again today is still counted as one new customer today.

④ non_customer — staff / rider / courier (headcount)

Every appearance of a person judged non-customer is counted, un-deduplicated.

Field	Meaning
count	Non-customer appearances this segment
breakdowns.person_type	Three types: staff, rider, courier

Operationally you subtract non-customers from raw flow; if a staff member enters/exits 20 times you subtract 20 — so this counts headcount, not persons, deliberately different from `final\_stats.count`.

Attribution rule: each appearance is attributed to its own segment by time, not concentrated into the earliest minute.

⚠ `person\_type` has no `customer` — customers are in `final\_stats`.

⚠ If this block is all-zero for a long time, the device most likely has person-type detection off — contact us to confirm; do not treat it as "this store has no staff".

⑤ dwell_stats — how long they stayed (headcount)

Field	Meaning
count	Customer entries this segment that have already been seen leaving
breakdowns.dwell_range	Dwell-duration buckets

⚠ Do not divide this by `final\_stats.count`. The two are not the same group of people: one is persons, the other is entry headcount, and those still in the store (or whose exit was not captured well) are excluded. The ratio is meaningless.

Attribution rule: attribute by entry time, not exit time. A customer who entered at 9:59 and left at 10:30 is counted in the 9:59 minute.

⚠ Count only customers; staff/rider in/out is not counted as dwell (otherwise a clerk staying all day would skew the distribution).

A customer who came 3 times that day and was seen leaving each time counts 3 times; whereas in `final\_stats` they are `count` 1 + `repeat` 2.

Bucket boundaries are fixed (minutes, left-closed right-open):

Bucket name	Range
0_1	under 1 minute
1_5	1 ~ 5 minutes
5_15	5 ~ 15 minutes
15_30	15 ~ 30 minutes
30_plus	over 30 minutes

⚠ Bucket boundaries are not configurable — if they were, data from different stores and periods could not be compared together.

2.4 Shape and Values of Buckets

The `breakdowns` of the four statistics blocks share the same shape: dimension key → array of detail rows.

Dimension key: a single dimension uses the dimension name directly (`height\_category` / `person\_type` / `dwell\_range`); a cross of two dimensions joins them with `|` (`age\_range|gender`).

Each row = the dimension values + `count`.

Only non-zero rows appear. Most of the 7 age bands × 3 genders combinations are 0; listing them all would bloat the message.

Row order is fixed (the table below), but read by row, do not index by position, and do not assume the number of rows or that combinations are complete.

Dimension	Values (i.e. row order)
height_category	above_threshold → below_threshold → unknown → not_measured
age_range	0_9 → 10_16 → 17_30 → 31_45 → 46_60 → 61_plus → unknown, last row not_measured
gender	male → female → unknown, last row not_measured
person_type	staff → rider → courier
dwell_range	0_1 → 1_5 → 5_15 → 15_30 → 30_plus

⚠ For `age\_range|gender`, "not measured" is one row where both keys are `not\_measured`:

```
{ "age_range": "not_measured", "gender": "not_measured", "count": 3 }
```

Do not write `gender: "unknown"` — that would fold "feature off" back into "measured but unrecognized", mixing the two classes when grouping by gender. `person\_type` and `dwell\_range` have no `not\_measured`.

On `height\_category`: the dividing line is an adjustable height threshold on the device (default 120 cm). The device reports only the conclusion; the message has no raw height, and the bucket name does not hard-code the number — if the site changes the threshold to 130, the bucket name will not lie along with it.

⚠ `above\_threshold` / `below\_threshold` are pure height bands, not adult/child. A short adult also falls into `below`. Age semantics are carried by `age\_range`.

2.5 Self-Check: These Numbers Should Add Up

```
end_ts_s = start_ts_s + 60
raw_stats.count = final_stats.count + final_stats.repeat + non_customer.count
← 7 = 4 + 1 + 2 [strict]
sum of dimension buckets = the block's count ← strict
raw_stats: 5+1+1 = 7    3+2+1+1 = 7
final_stats: 3+1 = 4    2+1+1 = 4 ← excludes repeat
non_customer: 1+1 = 2
dwell_stats: 2+1 = 3
raw_stats.count <= event_counts[stat_basis] ← converge to equal
dwell_stats.count <= final_stats.count + final_stats.repeat
```

The first two are strict equalities, no exceptions. People during feature-off are caught by the `not\_measured` bucket and will not leak outside the total. If they do not match, the device neither sent a value nor marked "not measured" — a data problem worth feeding back to us.

The last two are inequalities; right after reporting there may be a small gap that then catches up. Do not use them as hard validation to reject or alert.

⚠ Do not discard the message even if validation fails. Store by value and record an alert — discarding causes unrecoverable data loss, and the device's next revision would overwrite it anyway.

3. Why the Same Minute Arrives Multiple Times

3.1 revision — When This Version of the Numbers Was Computed

`revision` is a monotonic non-decreasing integer, whose value is the moment (Unix seconds) this version of the numbers was computed. You only need to compare magnitudes; you do not need to interpret its meaning.

When multiple messages for the same minute arrive, keep the one with the largest `revision` and discard the rest.

Do not assume it starts from 1, do not assume a step size, do not count it as "which version". The first time you receive a minute, it is already a large number — that is normal.

A side benefit: it is exactly the freshness of this data, and can be shown directly as "last updated at".

⚠ Receiving a 2nd or 3rd message for the same minute is normal operation, not a duplicate-delivery fault.

Why it keeps getting revised — three real scenarios:

Scenario	What happened
Exit seen	A customer who entered at 9:59 only leaves at 10:30. Only at that moment can the 9:59 minute's `dwell_stats` be filled
Recognized as same person	A person judged a new customer at 10:05 reappears at 10:40, photographed more clearly, and is recognized as the one who came at 9:10 ⇒ the 10:05 minute's `count` -1, `repeat` +1
Identity re-judged	Entered at 9:10 with no badge captured, judged a customer; returns at 10:40 with badge captured ⇒ re-judged as staff; all their appearances that day move from `final_stats` to `non_customer`, and every affected minute must be resent

Common point: later observations rewrite earlier conclusions. This is an inherent property of cross-time, cross-camera identity recognition, not incomplete data arrival — to truly stop changing you would have to wait until the business day ends.

⚠ So do not design it as "lock on first receipt". To judge whether a day has been finalized: that day receives no new messages (see 4.3).

3.2 Reset: The Day's Numbers Are Wholly Voided

When an on-site business reset happens, cameras are added/removed, or the statistical basis direction changes, the numbers already reported for that day become invalid and must be recomputed.

The expression is an ordinary message: the device sends, for each minute already reported that day, one all-zero message (with `revision` increased as usual), then the numbers recomputed under the new config are sent minute by minute.

```
{
  "schema_version": "1.0",
  "type": "interval_aggregate",
  "master_sn": "CAM-A01", "device_sns": ["CAM-A01", "CAM-B02"],
  "timezone": { "name": "Asia/Shanghai", "utc_offset_seconds": 28800, "is_dst":
false },
  "batch_date": "2026-03-04", "stat_basis": "enter",
  "revision": 1741050000,
  "interval_s": 60, "start_ts_s": 1741046400, "end_ts_s": 1741046460,
  "event_counts": { "enter": 0, "leave": 0, "pass_by": 0, "turn_back": 0 },
  "raw_stats": { "count": 0, "breakdowns": { "height_category": [] },
"age_range|gender": [] } },
  "final_stats": { "count": 0, "repeat": 0, "breakdowns": { "height_category":
[], "age_range|gender": [] } },
  "non_customer": { "count": 0, "breakdowns": { "person_type": [] } },
  "dwell_stats": { "count": 0, "breakdowns": { "dwell_range": [] } }
}
```

You do nothing: it is exactly the same shape as an ordinary message; replace the whole record by the rules of section 0.

Only the minutes actually reported that day are cleared; a minute with no one passing will not conjure a zero out of nowhere.

Reset is a low-frequency event (installation debugging, device add/remove, config adjustment), not a daily action.

Routine day-crossing needs no clearing: a new day is a new `batch_date`, a new set of keys, unrelated to yesterday.

⚠ Clearing plus resending takes a few minutes to finish. During this time the day's report is in a "half-cleared" state; seeing the day's numbers temporarily low is normal — just wait for it to finish. Avoid on-site adjustments during peak business hours.

⚠ This is the only scenario in the whole protocol where an "all-zero" message appears. During normal business a minute's numbers will not turn to 0 on their own — if someone really passed that minute, they only move among the three cells "new customer / repeat / non-customer", the total unchanged.

⚠ But a single cell going to zero is normal: a minute's `final_stats.count` can go from 3 to 0 — those three were all recognized as regulars who came earlier that day. Then `repeat` must be 3 and `raw_stats.count` is still 3. Seeing `final_stats.count: 0` with `raw_stats.count: 3` is not an anomaly.

4. Sending Cadence

4.1 Fixed One-Minute Segments

The segment width is fixed at 60 seconds and does not change with any config.

The in-progress minute is also reported, then revised as that minute completes — so the one you just received is not necessarily that minute's final value.

⚠ Thus `revision` increments for two reasons: the numbers were genuinely revised (3.1), and this minute has not finished yet. You need not distinguish; take the larger and overwrite, and do not infer data quality from how fast `revision` grows.

4.2 When Messages Appear

Sent as soon as computed; does not wait for the minute to finish.

A minute with no one passing produces no message: if all four directions had no one this minute, nothing is sent.

⚠ Therefore gaps in the segment sequence are normal, not lost messages. A `start_ts_s` never received = no one at the door that minute.

⚠ Do not use "no message received for a minute" to judge the device offline. Whether the device is online is the responsibility of the device-status heartbeat, which is a separate uplink. A normal heartbeat but no message that minute simply means no one that minute.

⚠ "All four statistics blocks are zero" does not mean no one: with `stat\_basis` = enter, a minute with only departures and no entries has `event\_counts.leave` > 0 while all four blocks are 0. This is a real message that must be received; discarding it would create a false gap in your exit curve.

After closing, the device goes completely silent and sends no messages.

4.3 Retry and Offline Tolerance

Sent one by one: the next is sent only after the previous is acknowledged.

On failure, retry with a fixed interval, retrying until success.

Only the latest version of each segment is sent; intermediate versions are not back-filled. If this minute's numbers are revised again while waiting to retry, the device overwrites the pending one with the new version rather than queueing both — so what you receive is always "the latest value at the moment the device sent it".

Thus `revision` will skip numbers — a segment jumps from ...520 straight to ...910; the intermediate versions existed but were not sent. This does not affect you (only compare magnitudes); do not treat it as lost and demand retransmission.

The backlog after reconnect = the number of segments that had data, and does not multiply with longer downtime.

The device keeps at most 2 days. Data still undelivered after 2 days is purged — these 2 days are your server's maximum offline tolerance. After recovery the device back-fills the backlog first, then converges to the final numbers.

There is no "this day is over" message. To judge whether a day has stabilized: that day receives no new messages. Whether the device is still alive is seen from the device-status heartbeat.

⚠ For a date that has gone quiet, correction messages may appear again (business-time config adjustment causes that day to be recomputed). Apply by the version rules as usual; do not alert just because "yesterday's date sent another message".

5. After You Have the Data

The numbers in this protocol can be used directly for reports, no secondary processing needed.

Common usages:

What you want	How to get it
Raw door entries	<code>event_counts.enter</code>
Exit / pass-by / turn-back volume	the other three keys of <code>event_counts</code>
Precise flow (deduplicated customer count)	<code>sum final_stats.count over segments</code>
Total customer headcount	<code>sum (final_stats.count + final_stats.repeat) over segments</code>
Repurchase / repeat rate	<code>final_stats.repeat ÷ (count + repeat)</code>
Non-customers to subtract	<code>non_customer.count</code>
Customer profile (age / gender / height)	<code>final_stats.breakdowns</code>
Visit headcount profile (un-deduplicated)	<code>raw_stats.breakdowns</code>

Dwell-duration distribution	dwell_stats.breakdowns
Any time granularity (5 min / hour / day)	directly add the corresponding numbers of adjacent segments

When aggregating to a coarser granularity, `final_stats.count`` can be added directly. A customer is counted into `count`` in only one segment that day, so adding the `count`` of several segments gives the "first-time-that-day" customer count over that period. For the day's deduplicated total customer count, sum all the day's segments' `count``.

⚠ But you cannot add across days: the deduplication scope is one business day; yesterday's and today's `count`` added do not equal "distinct people over two days".

6. Common Misuse Checklist

Misuse	Correct approach	See
Return a 200 with empty body	Response body must be non-empty, or the device judges failure and resends	0
Put <code>stat_basis`</code> in the primary key	It is only a descriptive field on the row; a direction change clears and recomputes the day	2.2
Alert on "duplicate delivery" when the same minute arrives multiple times	Revision is normal; take the larger by <code>revision`</code>	3.1
Reject / alert on an all-zero message	That is an on-site reset clearing the day's old numbers; replace the whole record as usual	3.2
Treat <code>revision`</code> as "which version" or assume it starts from 1	It is the moment this version was computed; only compare magnitudes	3.1
Return 4xx / 5xx for a late message with smaller <code>revision`</code>	Just discard the data, but still return 2xx + non-empty body	0
Merge <code>not_measured`</code> into <code>unknown`</code>	The former is feature off, the latter is unrecognized; coverage would be computed wrong	1.4
Assume turning off a detection switch makes the day's existing distribution disappear	The message describes facts at the moment people passed, not the current switch state	1.4 / 2.3 ④
Deduplicate yesterday's customers against today's <code>count`</code>	Deduplication scope is one business day; no cross-day deduplication	2.3 ③ / 5
Add <code>final_stats.count`</code> and <code>repeat`</code> as persons	The former is persons, the latter headcount; the sum is headcount	2.3 ③
Use <code>dwell_stats.count`</code> ÷ <code>final_stats.count`</code> as a ratio	The two are not the same group of people and are not comparable	2.3 ⑤
Use <code>non_customer`</code> as persons to subtract	It is headcount; a staff member in/out 20 times	2.3 ④

	subtracts 20	
Attribute dwell to exit time	Attribute to entry time	2.3⑤
Treat `final_stats.count: 0` as an anomaly	That minute's entries were all regulars from earlier that day, all in `repeat`	3.2
Discard "all four blocks zero" as no one	Could be a minute with only departures, `event_counts` non-zero	4.2
Treat gaps in the segment sequence as lost and actively demand retransmission	Minutes with no one passing produce no message	4.2
Use "no message for a minute" to judge device offline	Online status is the responsibility of the device-status heartbeat	4.2
Wait for a "this day is over" message	Does not exist; manifests as no more messages for that day	4.3
Use `event_counts[stat_basis] == raw_stats.count` as a hard validation	May differ right after reporting, then catch up	2.3① / 2.5
Infer a data problem from `revision` growing fast	Could also just be that this minute has not finished	4.1
Assume `breakdowns` row count is fixed or combinations complete	Only non-zero rows appear; read by row	2.4
Reverse-engineer `batch_date` from `start_ts_s`'s calendar day	Business day may cross midnight; the two are unrelated	1.2
Discard the message when time-zone name parsing fails	`name` may be in the form `"UTC-12:45"`; accept both	1.1